Compte rendu : TP 3

Réalisé par LOARI Moumouni Boukary

1. Configuration nginx (étapes 1 et 2)

Le serveur www.tp.local présente la chaîne **feuille + intermédiaire** (serveur_chaine.crt, jamais la racine, que le client possède déjà). Les fichiers sont installés dans /etc/nginx/tls (clé en mode 600, certificats et CRL en 644) et le nom est ajouté à /etc/hosts. La configuration est dans /etc/nginx/conf.d/tp.conf. sudo nginx -t renvoie « syntax is ok / test is successful » ; le rechargement se fait par sudo systemctl reload nginx, sans coupure. Une comparaison cmp a montré que la chaîne installée différait de celle construite ; elle a été réinstallée et openssl verify donne OK.

Directive	**Justification**
ssl_protocols TLSv1.2 TLSv1.3	Exclut SSLv3, TLS 1.0 et 1.1, obsolètes
ssl_ciphers 'ECDHE+AESGCM:ECDHE+CHACHA20'	TLS 1.2 limité à ECDHE (confidentialité persistante) et AEAD
ssl_session_tickets off	Les clés de tickets ne sont pas renouvelées : leur fuite déchiffrerait les sessions passées
ssl_verify_client on, ssl_verify_depth 2, ssl_client_certificate (racine)	Certificat client obligatoire, chaîne client → intermédiaire → racine ; le client envoie donc aussi l'intermédiaire (bob_chaine.crt)
ssl_crl (CRL de l'intermédiaire et de la racine)	Refuse les certificats révoqués ; une CRL par niveau est nécessaire
add_header Strict-Transport-Security "max-age=31536000" always	HSTS : HTTPS imposé pendant un an

2. Observation de la poignée de main (étape 3)

Scénario	**Résultat observé**
Bob, TLS 1.3	TLSv1.3, TLS_AES_256_GCM_SHA384, groupe X25519MLKEM768, Verification: OK, 200 OK : Bonjour CN=bob@tp.local
Bob, TLS 1.2 forcé	TLSv1.2, ECDHE-ECDSA-AES256-GCM-SHA384, Verification: OK, 200 OK

Scénario	Résultat observé
Sans certificat client	HTTP 400 « No required SSL certificate was sent »
Alice (révoquée)	HTTP 400 « The SSL certificate error » ; journal nginx : (23:certificate revoked)

Avec openssl s_client, l'option -cert n'envoie que le premier certificat du fichier : il faut ajouter -cert_chain inter/inter.crt, sinon nginx (qui ne connaît que la racine) répond 400 même pour Bob. curl envoie toute la chaîne de bob_chaine.crt. nginx ne renvoie pas d'alerte TLS à un client refusé : la poignée de main se termine, puis une page HTTP 400 est servie. Dans la capture Wireshark (capture_tls.pcapng, 32 paquets), la connexion **TLS 1.3** montre ClientHello (SNI www.tp.local) et ServerHello en clair, puis seulement des Application Data chiffrés : **aucun message Certificate n'est visible**. La connexion **TLS 1.2** montre au contraire le certificat du serveur et celui du client en clair.

3. Vérification de la configuration (étape 4)

Un client TLS 1.1 est refusé (no protocols available côté client, alerte 70 protocol version si on abaisse le niveau de sécurité). Le scan nmap --script ssl-enum-ciphers -p 443 a d'abord montré **13 suites TLS 1.2 dont 6 en CBC** (sans chiffrement authentifié) ; après ajout de ssl_ciphers et rechargement, il ne reste que **3 suites AEAD** en TLS 1.2 et les **3 suites AEAD** de TLS 1.3.

4. Supervision et renouvellement (étapes 5 et 6)

surveiller.py DOSSIER lit les .crt (chaînes comprises) et rend 0 (OK), 1 (alerte < 30 j) ou 2 (critique < 7 j), utilisable par un outil de supervision. Trois dossiers de test donnent les codes 0 (serveur.crt : OK, 89 j), 1 (moyen.crt émis pour 20 jours : ALERTE, 19 j) et 2 (court.crt émis pour 5 jours : CRITIQUE, 4 j). Sur /etc/nginx/tls : racine 3649 j (expire le 2036-10-27), serveur 89 j (2027-01-28), intermédiaire 1824 j (2031-10-29), tous OK, code 0. renouveler.sh renouvelle si l'expiration approche : nouvelle clé et CSR, signature, **vérification avant déploiement** (openssl verify, concordance clé/certificat), archivage de l'ancien, installation, nginx -t, reload, **retour arrière en cas d'échec**.

Scénario	Résultat
Certificat valide > 30 jours	« rien à faire », code 0
Renouvellement forcé (vrai nginx)	Série 1000 → 100B, verify OK, nginx -t OK, valable jusqu'au 29/01/2027, curl répond, ancien archivé

Scénario	Résultat
Échec simulé (NGINX_TEST=false)	« ancien certificat remis en place », code 1, série 100B avant et après, curl répond
Planification	Ligne cron quotidienne 17 3 * * * installée puis retirée (crontab -r)

Défaut trouvé et corrigé. Un premier test a montré la série 1009 au lieu de 1008 : le retour arrière ne faisait rien alors que le script affichait « retour à l'ancien certificat ». Cause : l'archive est en mode 700 et appartient à root, et [-d "$archive"] exécuté par un utilisateur ordinaire échouait. Correction : tests via sudo et vérification du résultat par cmp. Leçon : un test de retour arrière compare l'état réel, et un script se teste avec les droits réels de son utilisateur.

5. Questions de réflexion

1. **Pas de racine envoyée ?** Le client la possède déjà : une racine reçue du réseau ne prouverait rien, et elle alourdit chaque poignée de main.
2. **Tickets de session désactivés ?** Si la clé de ticket n'est jamais renouvelée, sa fuite permet de déchiffrer a posteriori le trafic enregistré, ce qui annule la confidentialité persistante d'ECDHE ; en échange on perd la reprise rapide.
3. **HSTS ?** Le navigateur n'utilise que HTTPS pendant max-age et ne permet pas d'ignorer une erreur de certificat : protection contre le *SSL stripping*. La première visite n'est protégée que par la liste de préchargement.
4. **Certificats de 100 jours (2027), 47 jours (2029) ?** Le renouvellement manuel devient impraticable : il faut automatiser (ACME, scripts), tenir un inventaire, superviser les expirations, renouveler bien avant l'échéance et pouvoir changer d'AC vite. Bénéfice : une clé volée sert moins longtemps.
5. **Chaîne incomplète ?** Certains clients la complètent (intermédiaire en cache ou téléchargée via l'extension AIA), d'autres (curl, OpenSSL, Java, applications mobiles) la refusent : le site marche dans le navigateur mais pas dans un script. Il faut toujours servir feuille et intermédiaires.
6. **Fuite de la clé du serveur :** confirmer et dater ; générer une **nouvelle** paire ; révoquer l'ancien certificat (keyCompromise) et republier les CRL ; émettre et déployer le nouveau (FORCE=1 ./renouveler.sh) puis redémarrer nginx ; chercher et corriger la cause ; informer et consigner l'incident. Grâce à ECDHE, le trafic passé reste indéchiffrable avec la seule clé du serveur.